Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»



З лабораторної роботи

з дисципліни « WEB-програмування »

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

Звіт до Лабораторної роботи №4

На тему: «Робота з циклами, функціями та функціональними виразами в JavaScript. Консоль браузера»

1. Мета і результати навчання

Мета роботи: Реалізувати та пояснити алгоритм аналізу послідовності вимірювань: від перевірки даних і циклів до декомпозиції на функції та синхронної передачі функції як аргументу.

Отримані практичні навички:

- Підключення зовнішнього JavaScript-скрипта до HTML-сторінки з використанням атрибута defer та діагностика його виконання у Browser DevTools -> Console.
- Підбір та практичне застосування циклів for, for...of, while та do...while відповідно до умов алгоритму; осмислене використання операторів break і continue.
- Побудова функцій з параметрами та оператором return; використання різних форм запису функцій (Function Declaration, Function Expression, Arrow Functions).
- Опрацювання концепцій Scope (область видимості), Hoisting (підняття) та передача функцій через зворотні виклики (callbacks).

2. Постановка задачі та вихідні дані

В рамках адаптивного проєкту **«Акустична варта»** необхідно розробити модуль js/monitoring.js, який проводить аналіз масиву вимірювань рівня шуму, відхиляє недостовірні дані, здійснює статистичні розрахунки, знаходить перші критичні точки й можливість відновлення, а також виводить підсумковий звіт у консоль браузера без зміни DOM-структури сторінки.

Вихідний масив вимірювань та параметри:

```
const noiseLevels = [
  42, 55, 61, 78, 83, 67, 49, 72, 91, 58, 64, 76,
  53, 47, 88, 69, 74, 82, 95, 51, 71, -1, 85, 63,
  79, 39, 141, 86
];

const warningLevel = 70;
const criticalLevel = 85;
```

```
const minValidLevel = 30;
const maxValidLevel = 130;
```

3. Структура проєкту

```
project/
├── index.html
├── css/
│   └── main.css
├── images/
└── js/
    └── monitoring.js
```

У файлі index.html підключення реалізовано в секції <head>:

```
<script src="./js/monitoring.js" defer></script>
```

4. Код програми (js/monitoring.js)

```
const noiseLevels = [
  42, 55, 61, 78, 83, 67, 49, 72, 91, 58, 64, 76,
  53, 47, 88, 69, 74, 82, 95, 51, 71, -1, 85, 63,
  79, 39, 141, 86
];

const warningLevel = 70;
const criticalLevel = 85;
const minValidLevel = 30;
const maxValidLevel = 130;

// 2. ДЕКОМПОЗИЦІЯ НА ФУНКЦІЇ

// Arrow Function з неявним поверненням
const formatDb = value => `${value} dB`;

// Arrow Function для перевірки валидності значення
const isValidLevel = (level, min = minValidLevel, max = maxValidLevel) =>
  Number.isFinite(level) && level >= min && level <= max;
```

```
// Function Expression: аналіз за категоріями з використанням циклу for...of та continue
const countByCategory = function(data, warning, critical) {
  const stats = {
    total: data.length,
    valid: 0,
    invalid: 0,
    normal: 0,
    warningCount: 0,
    criticalCount: 0,
    invalidDetails: []
  };

  let idx = 1;
  for (const level of data) {
    if (!isValidLevel(level)) {
      stats.invalid++;
      stats.invalidDetails.push({ position: idx, value: level });
      idx++;
      continue;
    }

    stats.valid++;
    if (level <= warning) {
      stats.normal++;
    } else if (level > warning && level <= critical) {
      stats.warningCount++;
    } else {
      stats.criticalCount++;
    }
    idx++;
  }
  return stats;
};

// Обчислення метрик: середнє, мінімум, максимум
function calculateMetrics(data) {
  let sum = 0;
  let count = 0;
  let min = null;
  let max = null;

  for (const level of data) {
```

```
    if (!isValidLevel(level)) continue;

    sum += level;
    count++;

    if (min === null || level < min) min = level;
    if (max === null || level > max) max = level;
  }

  const average = count > 0 ? sum / count : null;
  return { average, min, max };
}

// Function Declaration: пошук першого критичного значення з використанням for та
break
function findFirstCritical(data, criticalThreshold) {
  for (let i = 0; i < data.length; i++) {
    const level = data[i];
    if (!isValidLevel(level)) continue;

    if (level > criticalThreshold) {
      return {
        value: level,
        position: i + 1
      };
    }
  }
  return null;
}

// Пошук моменту відновлення за допомогою циклу while
function findRecovery(data, firstCriticalIndex, warningThreshold) {
  if (!firstCriticalIndex) return null;

  let currentIndex = firstCriticalIndex.position; // починаємо з наступного після
критичного
  let safeStreak = 0;

  while (currentIndex < data.length) {
    const level = data[currentIndex];

    if (isValidLevel(level) && level <= warningThreshold) {
      safeStreak++;
```

```
      if (safeStreak === 3) {
        return { position: currentIndex + 1, value: level };
      }
    } else {
      safeStreak = 0;
    }
    currentIndex++;
  }

  return null;
}

// Перевірка за допомогою циклу do...while
function findFirstValidDoWhile(data) {
  if (!data || data.length === 0) return null;

  let index = 0;
  let found = null;

  do {
    if (isValidLevel(data[index])) {
      found = data[index];
      break;
    }
    index++;
  } while (index < data.length);

  return found;
}

// Передавання алгоритму через параметр (Callback)
function processData(data, callback) {
  return callback(data, warningLevel, criticalLevel);
}

// 3. ФОРМУВАННЯ КОНСОЛЬНОГО ЗВІТУ
function generateConsoleReport() {
  console.group('Акустична варта -- звіт моніторингу');

  console.log(`Пороги: попередження > ${formatDb(warningLevel)}; критичний рівень >
${formatDb(criticalLevel)}`);

  const stats = processData(noiseLevels, countByCategory);
```

```
  const metrics = calculateMetrics(noiseLevels);
  const firstCritical = findFirstCritical(noiseLevels, criticalLevel);
  const recovery = findRecovery(noiseLevels, firstCritical, warningLevel);

  console.log(`Усього вимірювань: ${stats.total}`);
  console.log(`Коректних: ${stats.valid}`);
  console.log(`Некоректних: ${stats.invalid}`);

  // Формування підсумкової таблиці
  const summaryTable = [
    { 'Категорія': 'Без перевищення', 'Кількість': stats.normal },
    { 'Категорія': 'Попередження', 'Кількість': stats.warningCount },
    { 'Категорія': 'Критичні', 'Кількість': stats.criticalCount }
  ];
  console.table(summaryTable);

  const avgString = metrics.average ? metrics.average.toFixed(2).replace('.', ',') : 'null';
  console.log(`Середній рівень: ${avgString} dB`);
  console.log(`Мінімальний рівень: ${formatDb(metrics.min)}`);
  console.log(`Максимальний рівень: ${formatDb(metrics.max)}`);

  if (firstCritical) {
    console.warn(`Перше критичне вимірювання: ${formatDb(firstCritical.value)}, позиція
${firstCritical.position}`);
  } else {
    console.log('Перше критичне вимірювання: не знайдено');
  }

  if (recovery) {
    console.log(`Відновлення -- три безпечні вимірювання поспіль: позиція
${recovery.position}`);
  } else {
    console.log('Відновлення -- три безпечні вимірювання поспіль: не знайдено');
  }

  const exceededCount = stats.warningCount + stats.criticalCount;
  console.log(`Поріг ${warningLevel} dB перевищено у ${exceededCount}
вимірюваннях.`);

  if (stats.invalidDetails.length > 0) {
    const invalidLog = stats.invalidDetails
      .map(item => `позиція ${item.position} (${item.value} dB)`)
      .join(', ');
```

```
  console.error(`Некоректні дані: ${invalidLog}.`);
 }

 let conclusion = `Висновок: зафіксовано ${stats.criticalCount} критичні вимірювання; `;
 if (firstCritical && !recovery) {
  conclusion += 'після першого з них стійкого відновлення не виявлено.';
 } else if (firstCritical && recovery) {
  conclusion += `стійке відновлення зафіксовано на позиції ${recovery.position}.`;
 } else {
  conclusion += 'усі значення в межах норми.';
 }
 console.log(conclusion);

 console.groupEnd();
}

// Запуск обробки
generateConsoleReport();
```

5. Результати виконання у Console (DevTools)

Після завантаження сторінки в браузері та відкриття DevTools (F12 -> вкладка Console), сформовано такий підсумковий вивід:

▼ Акустична варта -- звіт моніторингу
Пороги: попередження > 70 dB; критичний рівень > 85 dB
Усього вимірювань: 28
Коректних: 26
Некоректних: 2

(index)	Категорія	Кількість
0	Без перевищення	13
1	Попередження	9
2	Критичні	4

Середній рівень: 68,38 dB
Мінімальний рівень: 39 dB

Максимальний рівень: 95 dB

⚠ Перше критичне вимірювання: 91 dB, позиція 9

Відновлення -- три безпечні вимірювання поспіль: не знайдено

Поріг 70 dB перевищено у 13 вимірюваннях.

❌ Некоректні дані: позиція 22 (-1 dB), позиція 27 (141 dB).

Висновок: зафіксовано 4 критичні вимірювання; після першого з них стійкого відновлення не виявлено.

6. Висновки

1. Під час виконання лабораторної роботи було освоєно підключення зовнішніх файлів JavaScript через <script src="..." defer>.
2. Застосовано різні види циклів:
    a. for...of — для підрахунку та фільтрації за категоріями;
    b. for — для пошуку першого критичного вимірювання із застосуванням break;
    c. while — для перевірки умови отримання трьох послідовних безпечних показників;
    d. do...while — для гарантованої первинної перевірки наявності хоча б одного коректного елемента.
3. Проведено декомпозицію вихідного коду на окремі функції з чітким розділенням обов'язків (обчислення повертаються через return, вивід здійснюється окремо).
4. Опрацьовано відмінності між Function Declaration (підтримує Hoisting), Function Expression (передається як параметр у вищорядкові функції) та Arrow Functions.
5. Навченося форматувати консольний звіт із використанням груп (console.group), таблиць (console.table), попереджень (console.warn) та логів помилок (console.error).